

Experiment 6

Aim: Orchestration of application components/microservices using Kubernetes and Docker containers.

1 What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform used to deploy, manage, scale, and maintain containerized applications.

It is especially useful when an application consists of multiple microservices or containers.

2 Why is Kubernetes Required?

Managing a few Docker containers manually is easy, but managing hundreds of containers becomes difficult.

Kubernetes helps with:

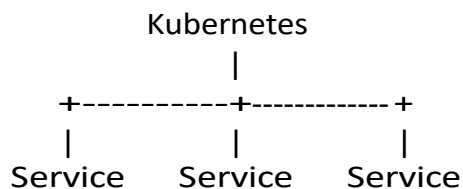
- Container deployment
- Container management
- Scaling applications
- Load balancing
- Service discovery
- Automatic recovery
- Rolling updates
- Resource management

Example

Without Kubernetes:

Container 1
Container 2
Container 3
Container 4
|
v
Manual Management

With Kubernetes:



| Container | Container | Container

Container Orchestration

Container orchestration means automatically managing the lifecycle of containers. It includes:

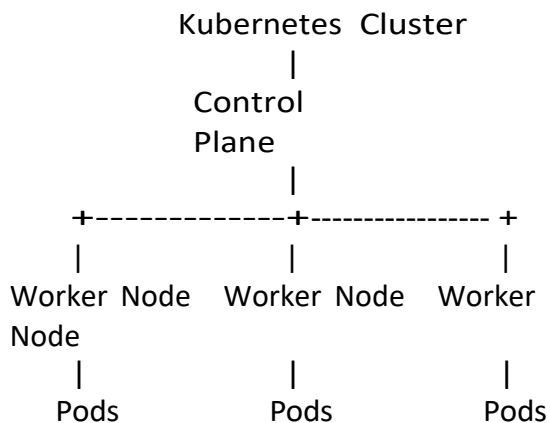
- Starting containers
- Stopping containers
- Restarting failed containers
- Scaling containers
- Networking containers
- Distributing workloads

Kubernetes performs these tasks automatically according to the desired configuration.

3 Kubernetes Cluster

A Kubernetes cluster is a group of machines used to run containerized applications. It mainly contains:

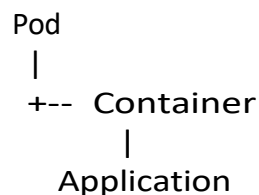
- **Control Plane:** Responsible for managing the Kubernetes cluster.
- **Worker Nodes:** Machines where application containers actually run.



4 Pod

A Pod is the smallest deployable unit in Kubernetes.

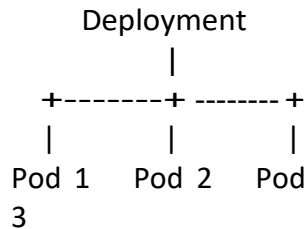
A Pod usually contains one application container, although it can contain multiple closely related containers.



5 Deployment

A Deployment manages Pods and ensures that the required number of application instances are running.

For example, if we require 3 replicas:

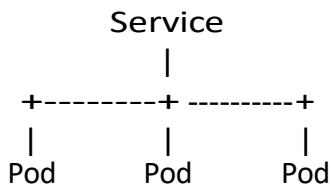


If one Pod fails, Kubernetes can create another Pod to maintain the desired number.

6 Service

A Service provides a stable way to access a group of Pods.

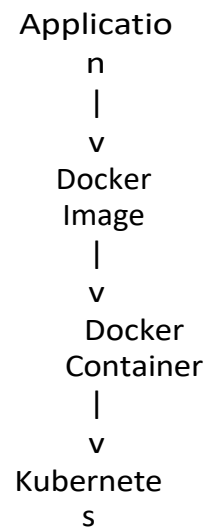
Since Pods can be created or removed dynamically, their IP addresses may change. A Service provides a consistent network endpoint.



7 Kubernetes with Docker

Docker is used for building and running containerized applications, while Kubernetes is used for orchestrating and managing containers at scale.

Relationship



|
v
Manage / Scale / Network /
Recover

8 Practical Screenshots and Descriptions

1. Project Structure

```
yash@YashSuthar MINGW64 ~
$ tree k8s-microservices
k8s-microservices/
├── user-service/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
├── message-service/
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── k8s/
    ├── user-deployment.yaml
    ├── user-service.yaml
    ├── message-deployment.yaml
    └── message-service.yaml
```

Screenshot 1: Project Structure

The project contains two microservices, their Dockerfiles, and Kubernetes Deployment and Service manifests.

2. user-service/app.py

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return "Hello from User Service!"
8
9 if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
```

Screenshot 2: User Service Application

The user-service Flask application returns "Hello from User Service! on port exttt5000.

3. message-service/app.py

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def home():
7     return "Hello from Message Service!"
8
9 if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)
```

Screenshot 3: Message Service Application

The message-service Flask application returns "Hello from Message Service!" on port 5000.

4. Dockerfile (Same for both services)

```
1 FROM python:3.9-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install -r requirements.txt
7
8 COPY app.py .
9
10 CMD ["python", "app.py"]
```

Screenshot 4: Dockerfile for Both Services

The Dockerfile uses a Python base image, installs dependencies, and starts app.py.

5. Build Docker Images

```
yash@YashSuthar MINGW64 ~/k8s-microservices
$ docker build -t user-service ./user-service
[+] Building 2.4s (8/8) FINISHED
=> => naming to docker.io/library/user-service
Successfully built user-service

yash@YashSuthar MINGW64 ~/k8s-microservices
$ docker build -t message-service ./message-service
[+] Building 2.1s (8/8) FINISHED
=> => naming to docker.io/library/message-service
Successfully built message-service
```

Screenshot 5: Building Docker Images

The docker build commands successfully create images for both microservices.

6. Verify Docker Images

```
yash@YashSuthar MINGW64 ~/k8s-microservices
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
user-service	latest	a1b2c3d4e5f6	10 seconds ago	158MB
message-service	latest	b4c5d6e7f8a9	12 seconds ago	158MB
python	3.9-slim	7d1a3b5c9e2f	2 weeks ago	158MB

Screenshot 6: Verifying Docker Images

The docker images output lists both service images and the Python base image.

7. Kubernetes Manifests (Example: user-deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
        - name: user-service
          image: user-service:latest
          ports:
            - containerPort: 5000
```

user-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: user-service
spec:
  selector:
    app: user-service
  type: NodePort
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30001
```

Screenshot 7: Kubernetes Deployment and Service Manifests

The manifests show two user-service replicas and the intended mapping from node port 30001 to container port 5000.

8. Apply Kubernetes Manifests

```
yash@YashSuthar MINGW64 ~/k8s-microservices/k8s
$ kubectl apply -f user-deployment.yaml
deployment.apps/user-deployment created

yash@YashSuthar MINGW64 ~/k8s-microservices/k8s
$ kubectl apply -f user-service.yaml
service/user-service created

yash@YashSuthar MINGW64 ~/k8s-microservices/k8s
$ kubectl apply -f message-deployment.yaml
deployment.apps/message-deployment created

yash@YashSuthar MINGW64 ~/k8s-microservices/k8s
$ kubectl apply -f message-service.yaml
service/message-service created
```

Screenshot 8: Applying Kubernetes Manifests

The kubectl apply -f commands create Deployments and Services for both microservices.

9. Check Running Pods

```
yash@YashSuthar MINGW64 ~/k8s-microservices
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
user-deployment-5d7c9f6d7-abc12	1/1	Running	0	20s
user-deployment-5d7c9f6d7-def34	1/1	Running	0	20s
message-deployment-7f9d6c8d6-ghi56	1/1	Running	0	18s
message-deployment-7f9d6c8d6-jkl78	1/1	Running	0	18s

Screenshot 9: Checking Running Pods

The kubectl get pods output shows four running, ready Pods with zero restarts.

10. Check Services

```
yash@YashSuthar MINGW64 ~/k8s-microservices
$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	5m
user-service	NodePort	10.96.214.5	<none>	5000:30001/TCP	1m
message-service	NodePort	10.96.125.3	<none>	5000:30002/TCP	1m

Screenshot 10: Checking Kubernetes Services

The two NodePort Services expose the user and message services on ports 30001 and 30002, respectively.

11. Test the Services (Using Minikube IP)

```
yash@YashSuthar MINGW64 ~
$ minikube ip
192.168.49.2

yash@YashSuthar MINGW64 ~
$ curl http://192.168.49.2:30001
Hello from User Service!

yash@YashSuthar MINGW64 ~
$ curl http://192.168.49.2:30002
Hello from Message Service!
```

Screenshot 11: Testing the Services Using the Minikube IP

Requests to the Minikube IP on ports 30001 and 30002 return the respective service greetings.